



MLOps for Traditional Machine Learning

By

Chin Sek Yi (A0236536B)

Supervisor:

Markus Kirchberg

DSA4288 Honours Project in Data Science and Analytics

Department of Statistics and Data Science

National University of Singapore

2025/2026

Summary

In academic data science settings, projects typically focus on model development without considering industry deployment practices. This creates challenges when transitioning from academic research to industry applications, where reproducibility, collaboration, and integration with software engineering and DevOps practices are essential. This project addresses this gap by implementing a comprehensive MLOps pipeline that demonstrates how to bridge academic machine learning with industry-ready deployment practices.

The project implements a complete MLOps workflow using fraud detection as the use case with the PaySim synthetic financial transaction dataset. Fraud detection was selected because fraud patterns change rapidly, requiring robust MLOps infrastructure for continuous experimentation, testing, and deployment. The solution integrates DVC for data versioning, MLflow for experiment tracking, FastAPI for model serving, Streamlit for user interfaces, and GitHub Actions for CI/CD automation.

The key original contributions include architectural decision-making for multi-environment infrastructure design. Through iterative development across four distinct architectural plans, I systematically evaluated trade-offs between centralised versus distributed MLflow servers, single EC2 versus multi-EC2 deployments, and container orchestration strategies. The final architecture features three isolated environments (development, staging, production), each with dedicated MLflow tracking servers and separate backend/frontend EC2 instances. This

design emerged from solving complex challenges around service connectivity, resource allocation, and environment isolation within time and budget constraints.

The implementation achieved significant operational improvements, reducing experiment setup time by 95% from five hours to under five minutes, while eliminating manual logging errors through automated pipelines. The project provides a practical MLOps template that others can follow, demonstrating good practices for experiment tracking, model promotion, containerisation, and CI/CD automation. This work offers valuable insights for similar implementations, showing how to make informed architectural decisions when building production-ready MLOps systems using open-source tools and cloud infrastructure.

The project repository can be found in <https://github.com/ChinSekYi/mlops-fyp>.

Table of Contents

<i>MLOps for Traditional Machine Learning</i>	1
<i>Summary</i>	2
List of Figures	8
List of Tables	9
<i>Chapter 1 Introduction</i>	10
Chapter Overview	10
1.1 Problem Statement	10
1.2 Objectives	11
1.3 Project Scope and Repository	11
<i>Chapter 2 Data Science Component</i>	12
Chapter Overview	12
2.1 Business and Data Understanding	13
2.1.1 Business Context	13
2.1.2 Dataset description	14
2.2 Dataset Preparation	15
2.2.1 Data Cleaning and Transformation	15
2.3 Exploratory Data Analysis (EDA)	17
2.4 Model Development and Training	17
2.4.1 Model Evaluation and Results	18
2.4.2 Interpretation of Results	19
2.5 From Notebook to Production: Workflow Limitations	19

<i>Chapter 3 MLOps Architecture and Core Pipeline Implementation.....</i>	<i>21</i>
Chapter Overview	21
3.1 Introduction to MLOps.....	21
3.2 Core MLOps Pipeline Components	22
3.2.1 Data Access and Versioning (DVC).....	23
3.2.2 Model Training and Experiment Tracking (MLflow)	25
3.2.3 Model Registry and Promotion (MLflow)	30
3.2.4 Model Deployment and Serving	32
3.3 Pipeline Integration and Workflow.....	37
3.3.1 From Components to Integrated Pipeline	37
3.3.2 Automated Pipeline Execution	37
3.3.3 Workflow Benefits and Validation	38
3.3.4 Performance Validation	39
Closing Remarks for Chapter 3.....	39
<i>Chapter 4 Infrastructure</i>	<i>40</i>
Chapter Overview	40
4.1 EC2 Infrastructure for Services	41
4.1.1 MLflow Tracking Server EC2	41
4.1.2 Backend + Frontend EC2	42
4.1.3 Connectivity	43
4.1.4 Roles and Workflow	44
4.2 Containerisation and Orchestration	46

4.2.1 Purpose	46
4.2.2 Implementation	46
4.2.3 Orchestration	47
4.3 Environment Simulation (Dev, Staging, Prod)	47
4.3.1 Purpose	47
4.3.2 Key Points	48
4.3.4 Backend + Frontend Configuration	49
4.4 Continuous Integration, Deployment, and Testing	51
4.4.1 Definition	51
4.4.2 Why CI/CD matters in MLOps	51
4.4.3 CI/CD Pipeline in this project	52
4.4.4 Explanation of the CI/CD Workflow	53
Closing Remarks for Chapter 4	56
<i>Chapter 5 Evaluation and Analysis</i>	<i>57</i>
Chapter Overview	57
5.1 Operational Benefits and Process Improvements	58
5.2 Project Evolution and Architecture Iterations	59
5.2.1 Plan A: Single EC2 Multi-Container Approach	60
5.2.2 Plan B: Multi-EC2 Environment Separation	61
5.2.3 Plan C: Centralised MLflow Server with Environment Separation	62
5.2.4 Plan D: Production Multi-Environment Architecture (Final Implementation)	63
5.3 Challenges faced	65

5.4 Limitations and Future Work	66
<i>Chapter 6 Conclusion</i>	<i>68</i>
6.1 Summary	68
6.2 Key Achievements	68
6.3 Project Contribution	68
6.4 Future Potential	69
6.5 AI Tool Declaration.....	69
<i>Chapter 7 References</i>	<i>70</i>
<i>Changelog</i>	<i>70</i>

List of Figures

Figure 2.1: Data Science Lifecycle

Figure 3.1: Data pipeline with DVC

Figure 3.2: MLflow experiment page showing one run containing four nested runs

Figure 3.3: Detailed view of a nested run in MLflow

Figure 3.4: Side-by-side comparison of ML model performance

Figure 3.5: Architecture of the MLflow tracking server

Figure 3.6: MLflow UI showing registered models with versions

Figure 3.7: FastAPI backend data flow architecture

Figure 3.8: Complete MLflow, backend, and frontend architecture

Figure 3.9: Streamlit UI showing sample input and prediction

Figure 3.10: Overview of all components covered so far

Figure 4.1: MLflow Tracking Server with SSH tunnelling

Figure 4.2: Backend-Frontend EC2 Architecture

Figure 4.3: Model training workflow across environments

Figure 4.4: EC2 architecture across all environments

Figure 4.5: CI/CD workflow overview

Figure 5.1: Plan A - Single EC2 Multi-Container Approach

Figure 5.2: Plan B - Multi-EC2 Environment Separation

Figure 5.3: Plan C - Centralised MLflow Server

Figure 5.4: Plan D - Final Production Architecture

List of Tables

Table 2.1: PaySim Dataset Attributes

Table 2.2: Sample PaySim Transaction Record

Table 2.3: Model Performance Results

Table 3.1: Comparison of Data Versioning Tools

Table 3.2: MLOps Solutions to Traditional Data Science Challenges

Table 4.1: Service Ports

Table 4.2: MLflow Server Configuration Across Environments

Table 4.3: Backend + Frontend Configuration Across Environments

Table 4.4: CI/CD Pipeline Configuration and Triggers

Chapter 1 Introduction

Chapter Overview

The key sections are:

- 1.1. Problem statement
- 1.2. Objectives
- 1.3. Project Scope and Repository

1.1 Problem Statement

Machine learning projects often stop at model development, with little consideration for how models will be deployed, monitored, or maintained in production environments. This creates a significant gap between academic research and industry applications, leading to unreliable systems and wasted effort when models cannot be operationalised effectively.

Traditional notebook-based workflows lack reproducibility, version control, and automated deployment capabilities essential for production systems. Data scientists frequently struggle with manual model deployment processes, inconsistent environments, and difficulty collaborating on model iterations, resulting in models that perform well in development but fail in real-world applications.

1.2 Objectives

To bridge the gap between academic machine learning and industry deployment by implementing a complete, production-ready MLOps pipeline that demonstrates how models can be made reproducible, deployable, and continuously maintainable.

1.3 Project Scope and Repository

This project implements MLOps practices using fraud detection as a use case, leveraging the PaySim synthetic financial transaction dataset. The scope encompasses data versioning, experiment tracking, model registry, containerised deployment, and multi-environment orchestration across development, staging, and production environments.

Complete implementation available: <https://github.com/ChinSekYi/mlops-fyp>

Chapter 2 Data Science Component

Chapter Overview

The key sections are:

- 2.1 Business and Data Understanding
- 2.2 Dataset Preparation
- 2.3 Exploratory Data Analysis (EDA)
- 2.4 Model Development and Training
- 2.5 From Notebook to Production: Workflow Limitations

This chapter presents the data science lifecycle, shown in figure 2.1, as typically carried out by data scientists. It includes a series of steps such as data understanding, exploratory data analysis (EDA), model development and evaluation. To illustrate this lifecycle, we adapt a publicly available Kaggle project titled “Transaction Fraud Detection” by Ben Roshan (2021).

The objective is to demonstrate a typical data science workflow that can be performed in a Jupyter Notebook on a local machine, requiring minimal setup of tools. The notebook for this component can be found in `/notebooks/kaggle-paysim-notebook.ipynb`.

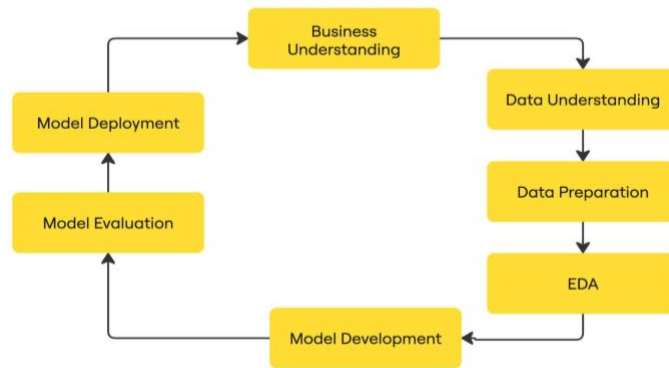


Figure 2.1: Data Science Lifecycle

2.1 Business and Data Understanding

2.1.1 Business Context

With the increase of digital use, it becomes a daily thing to transact money digitally, to accounts locally or globally, for a variety of reasons such as purchasing products, sending money to family, payments, or investments etc. This increase in digital usage has also led to a rise in fraudulent activities, where fraudsters attempt to impersonate legitimate users for money laundering. Fraudulent transactions can lead to significant financial loss. According to a global survey by Mastercard (2024), "75% of global consumers say they are more concerned about the impact of cybersecurity risks than they were two years ago." This highlights the urgent need for enhanced fraud detection systems that can effectively identify and prevent suspicious transactions in real-time.

2.1.2 Dataset description

To address this fraud detection challenge, this project utilises the PaySim synthetic dataset created by Lopez Rojas et al. (2016) as part of the research project "Scalable Resource-Efficient Systems for Big Data Analytics." The PaySim simulator models realistic transaction patterns based on aggregated data from a mobile money service provider, allowing researchers to develop and test fraud detection algorithms without exposing sensitive financial data.

Dataset Overview:

- Rows: 6,362,620 rows
- Columns: 10 numerical, 1 categorical
- Time Frame: 30 simulated days

Attribute	Description
step	Maps a unit of time in hours; total steps = 744 (30 days simulation).
type	Transaction type: CASH-IN, CASH-OUT, DEBIT, PAYMENT and TRANSFER.
amount	Transaction amount in local currency.
nameOrig	Originator of the transaction.
oldbalanceOrg	Initial balance of originator.
newbalanceOrig	New balance of originator after transaction.
nameDest	Recipient of the transaction
oldbalanceDest	Initial balance of recipient (merchants with "M" prefix excluded)
newbalanceDest	New balance of recipient (merchants with "M" prefix excluded).
isFraud	Fraudulent transactions by simulated agents (in PaySim simulation).
isFlaggedFraud	Flagged transactions attempting transfers > 200,000.

Table 2.1: PaySim Dataset Attributes

step	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
544	CASH_OUT	412668.39	C84071102	412668.39	0.0	C1576697216	1445047.47	1857715.86	1	0

Table 2.2: Sample PaySim Transaction Record

2.2 Dataset Preparation

Following the business and data understanding stage, the next step involves preparing the dataset for model development. Since this stage focuses on model experimentation and development, the full PaySim dataset (over 6 million rows) is not used to avoid excessive computational costs and long processing times. Instead, a smaller subset, `paysim-dev.csv` containing 4,000 sampled rows, is used. In the project repository, the `/data/raw` folder is intentionally left empty because the raw dataset is too large to be uploaded to GitHub. Instead, it is stored remotely on AWS s3 and can be pulled to local machines when required for development or testing. Details on how to retrieve the dataset from s3 can be found in the User Guide within the repository.

2.2.1 Data Cleaning and Transformation

To prepare the data for model training, several preprocessing steps were performed to ensure data quality, consistency, and suitability for supervised learning tasks. The following summarises the key cleaning and transformation steps:

- Numerical features were standardised using StandardScaler to normalize feature magnitudes.
- Categorical column type was one-hot encoded to represent transaction types as binary vectors.
- Text columns (nameOrig, nameDest) were tokenized into nameOrig_token and nameDest_token to provide a numerical representation of account identifiers.
- Unnecessary columns isFlaggedFraud, nameOrig, nameDest were dropped.
- Class imbalance handling: The severe class imbalance (~0.13% fraudulent transactions) was addressed using SMOTENC, which oversamples the minority class while considering both categorical and numerical features. This ensures that models are trained on a balanced dataset, reducing bias toward the majority class and improving detection of fraudulent transactions.

These transformations ensured that the input dataset was clean, standardised, and structured appropriately for downstream model training and evaluation.

Once the dataset was cleaned and transformed, the next step involved conducting Exploratory Data Analysis (EDA) to better understand the data distribution, identify potential patterns, and uncover insights that could guide feature selection and model development.

2.3 Exploratory Data Analysis (EDA)

EDA was performed to understand the dataset's composition and key characteristics before model training. The most notable finding was the high class imbalance in the raw data.

Although this imbalance was mitigated during preprocessing, it informed subsequent design decisions, including the choice of evaluation metrics that account for remaining imbalances (e.g., F1-score and Recall). Since the main objective of this project is to design and implement an MLOps pipeline rather than to perform in-depth statistical exploration, the EDA stage here is kept concise and primarily serves to validate dataset integrity and confirm key data characteristics relevant for model development. For a more detailed exploratory analysis, readers may refer to the original Kaggle notebook by Ben Roshan (2021), which this work adapts.

2.4 Model Development and Training

Building on the insights from EDA, four machine learning models were trained as part of the baseline workflow:

- Logistic Regression
- Decision Tree Classifier
- Gaussian Naive Bayes
- Random Forest Classifier

All models were trained using the prepared paysim-dev.csv dataset after the preprocessing steps described in the previous section. The goal at this stage was to establish a baseline understanding of model behaviour and to identify which algorithms perform best for the fraud detection task, before integrating the models into the automated MLOps pipeline described in the next chapter.

2.4.1 Model Evaluation and Results

After training the four baseline machine learning models on the paysim-dev.csv dataset, their performance was evaluated using Test Accuracy, F1-score, Precision, and Recall. The results are summarized in the table below:

Model name	Test Accuracy	F1 score	Precision	Recall
Logistic Regression	0.8417	0.8578	0.7702	0.9679
Decision Tree Classifier	0.9775	0.9732	0.9732	0.9814
Gaussian Naive Bayes	0.7867	0.8212	0.7000	0.9932
Random Forest Classifier	0.9783	0.9784	0.9609	0.9966

Table 2.3: Model Performance Results

The results indicate that ensemble-based models, particularly the Random Forest Classifier, achieved the highest F1-score and Recall, demonstrating strong performance in detecting

fraudulent transactions. These results provide a benchmark for the models before they are incorporated into the production-ready MLOps pipeline.

2.4.2 Interpretation of Results

The Random Forest Classifier achieved the highest overall performance, with a Test Accuracy of 97.83% and F1-score of 0.9784, indicating strong predictive ability for both classes. The Decision Tree Classifier was the second-best performer, while Logistic Regression and Gaussian Naive Bayes performed comparatively worse, particularly in terms of precision, suggesting a higher rate of false positives.

The use of SMOTENC during the data preprocessing stage addressed the class imbalance in the training dataset, allowing models to learn patterns for both fraudulent and non-fraudulent transactions and reducing bias toward the majority class.

Even though class imbalance is handled during preprocessing, F1-score and Recall remain important evaluation metrics, as the test or unseen data may still be imbalanced. Additionally, the financial impact of false negatives is high, so prioritizing recall ensures that fraudulent transactions are detected as reliably as possible.

2.5 From Notebook to Production: Workflow Limitations

Traditional notebook-based workflows present significant operational challenges that justify MLOps adoption:

1. **Experiment Tracking Challenges:** Tracking metrics across multiple models, preprocessing steps, and hyperparameter configurations is challenging. With numerous combinations of features, preprocessing methods, hyperparameters, and models, it becomes difficult to identify which configuration produces which results. Data scientists often resort to spreadsheets such as Microsoft Excel to record parameters and metrics, but this approach is manual, error-prone, and not easily reproducible.
2. **Computational Constraints:** Training models locally can be computationally intensive and time-consuming, highlighting the need for dedicated compute environments to support large-scale model training.
3. **Deployment Barriers:** Models trained and stored locally within notebooks cannot be readily deployed for use by others. Data scientists must transition from notebook-based experimentation to modular, production-ready scripts that define their data pipelines. Models should also be stored in centralised repositories, enabling inference and deployment through APIs.

These challenges highlight the need for MLOps, a structured framework that bridges the gap between data science experimentation and production deployment. The next chapter introduces MLOps concepts and demonstrates how this project applies them to create a reproducible, scalable, and maintainable machine learning workflow.

Chapter 3 MLOps Architecture and Core Pipeline

Implementation

Chapter Overview

This chapter introduces the motivation for adopting MLOps practices in this project and introduce tools and components that address the limitations from the Data Science workflow in Chapter 2.

The key sections are:

3.1 Introduction to MLOps

3.2 Core MLOps Pipeline Components

3.3 Pipeline integration and Workflow

3.1 Introduction to MLOps

Machine Learning Operations (MLOps) is a set of practices that combines principles from DevOps, data engineering, and machine learning to manage the end-to-end ML lifecycle—from data preparation and model training to deployment and monitoring.

Traditional data science workflows, often conducted in Jupyter notebooks, lack the structure needed for collaboration, reproducibility, and deployment. Organisations can either build their MLOps infrastructure from scratch or adopt third-party solutions.

- Third-party tools are faster to set up but often costly and less customisable.
- Self-built systems require more effort initially but offer greater flexibility and potential cost savings.

In this project, an open-source, self-built MLOps pipeline was implemented to simulate real-world practices while remaining cost-effective and fully customisable. The architecture integrates tools such as DVC, MLflow, FastAPI, and Streamlit, supported by Docker and AWS EC2 instances.

The project incorporates selected practices from Google Cloud's MLOps Maturity Levels 0, 1, and 2 (Google Cloud, 2021), ensuring that it provides a holistic yet practical view of MLOps development within the project's timeframe and scope.

3.2 Core MLOps Pipeline Components

This section introduces the key components of the MLOps pipeline, each of which corresponds to a specific stage of the data science lifecycle. The main components are:

1. Data Access and Versioning (DVC)
2. Model Training and Experiment Tracking (MLflow)

3. Model Registry (MLflow)
4. Model Deployment and Serving (FastAPI and Streamlit)

3.2.1 Data Access and Versioning (DVC)

The first core MLOps component focuses on managing the data itself: where it is stored, how it is versioned, and how reproducibility is ensured. Just as Git enables version control for code, similar tools exist for datasets and models. While Git is effective for source code, it has a file size limit of 100 MB per file, making it unsuitable for dataset tracking. To address this limitation, several tools have been developed. The table below summarises key data versioning solutions:

Tool	Introduced	Key Functionality	Limitation
Git-LFS	2015	Extension of Git for large binary files.	Slow push performance and dependency on external LFS servers.
LakeFS	2020	Versions unstructured /semi-structured data in data lakes (e.g., S3, GCS).	No native interface for viewing data; designed for large-scale enterprise data lakes.
Oxen.ai	2023	Fast, ML-optimised data versioning with collaboration features and web interface.	Emerging ecosystem; requires hosted service.
DVC	2016	“Git for data and models” — integrates with Git to track data, models, and pipelines.	Slower for very large pushes but well-established and lightweight.

Table 3.1: Comparison of Data Versioning Tools

DVC was selected for this project because it is open source, widely adopted, and integrates seamlessly with Git. The typical DVC workflow is as follows:

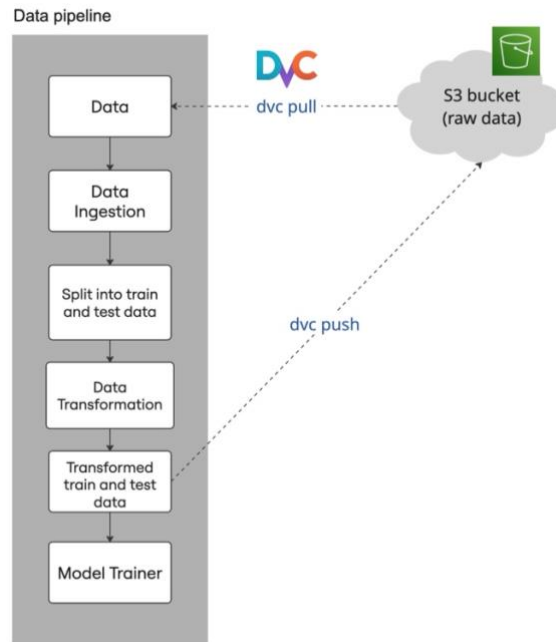


Figure 3.1: Data pipeline with DVC

1. Data Retrieval: dvc pull retrieves raw data from S3 storage into the development environment.
2. Data Upload: Processed datasets are versioned and pushed back to S3 using dvc push.
3. Version Tracking: Each dataset version is tracked via .dvc metadata files committed to Git.

For setup instructions and additional commands, refer to the developer documentation:

[docs/Developer_Guide/data-management.md](#).

3.2.2 Model Training and Experiment Tracking (MLflow)

3.2.2.1 Overview of Model Training

The model training pipeline encompasses several steps: model selection, training, hyperparameter tuning, and evaluation using appropriate metrics. This process enables models to learn patterns from the training data and make accurate predictions on unseen data.

In the PaySim fraud detection use case, each MLflow run trains four machine learning models on the same dataset to classify transactions as fraudulent or legitimate. As highlighted in Chapter 2 (*From Notebook to Production: Workflow Limitations*), one of the biggest challenges in data science is tracking experiments effectively. With numerous combinations of preprocessing methods, features, hyperparameters, and model choice, it becomes difficult to trace which configuration produced which result. Without a structured system, data scientists often rely on ad-hoc documentation (e.g., spreadsheets or local notes), which leads to errors, poor reproducibility, and limited collaboration.

An experiment tracking platform overcomes these issues by ensuring that:

- All preprocessing artefacts, model configurations, and hyperparameters are recorded.
- Experiments are fully reproducible and comparable.
- Team members can log, visualise, and analyse experiments in a centralised interface.

Common open-source and commercial tools for experiment tracking include MLflow, neptune.ai, Weights & Bias, Comet and DVC. For this project, MLflow was selected because it is open-source, widely adopted, and suitable for both individual and collaborative workflows.

3.2.2.2 Experiment and Run Tracking

In MLflow, each experiment is a top-level container. Within a single run, multiple models can be trained as nested runs. For the PaySim fraud detection experiment, one run contained four nested runs, one for each model type. Each nested run logs metrics, hyperparameters, and artefacts.

- Run configuration for reference: `/env/.env.machine.example`
- Model training implementation: `/src/components/model_trainer/`

Example run configuration in `.env`:

```
EXPERIMENT_NAME=fraud_detection_dev3  
  
RUN_NAME=paysim-dev-machine-v1  
  
DATASET_NAME=paysim-dev.csv  
  
ALGO_TYPE=traditional-ml  
  
REGISTERED_MODEL_NAME=dev.fraud-detection-model
```

Within this run, all four ML models described in Chapter 2 were trained, and their corresponding metrics and artefacts were logged. Users can inspect individual nested runs in the MLflow UI to view detailed logs, including metrics, hyperparameters, and artefact paths.

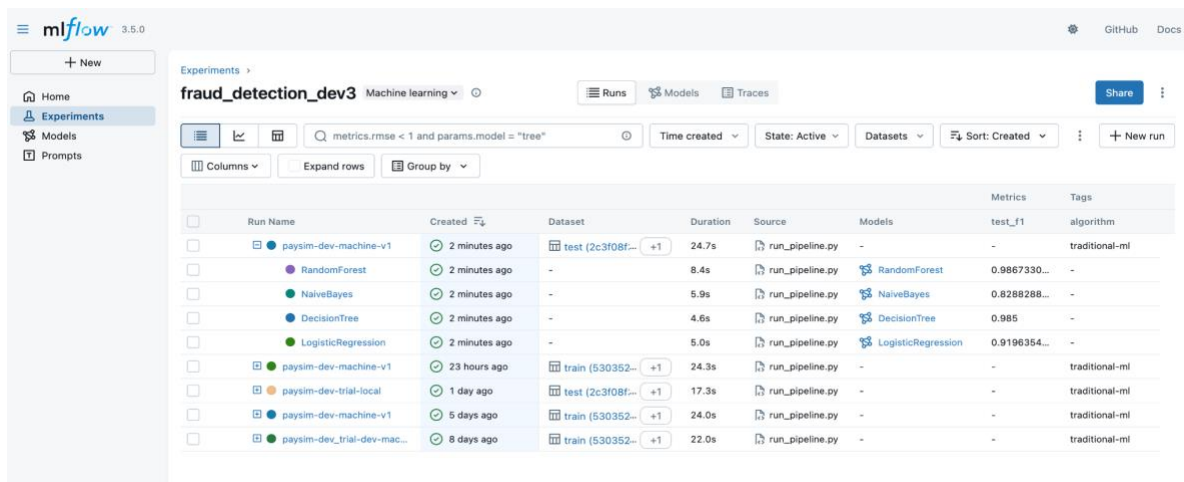


Figure 3.2: MLflow experiment page showing one run containing four nested runs, each corresponding to a different ML model.

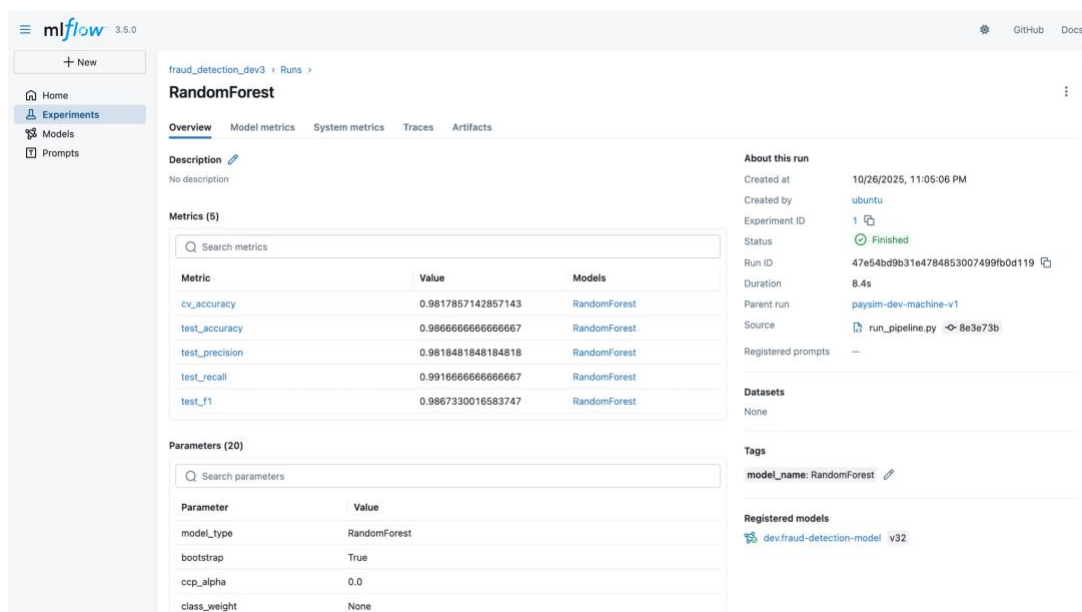


Figure 3.3: Detailed view of a nested run in MLflow, displaying hyperparameters, metrics, and logged artefacts for a specific model.

After logging all nested runs in MLflow, the next step is to compare the models' performance.

In Chapter 2, data scientists had to rely on spreadsheets or ad-hoc notes to track model

performance, which made identifying the best configuration error-prone and time-consuming. MLflow overcomes these limitations by providing a centralised comparison view.

For the `fraud_detection_dev3` experiment:

- The Random Forest model achieved the highest score for accuracy, precision, and F1-score.
- The Naïve Bayes model achieved lower accuracy, precision, and F1-score but had the highest recall.

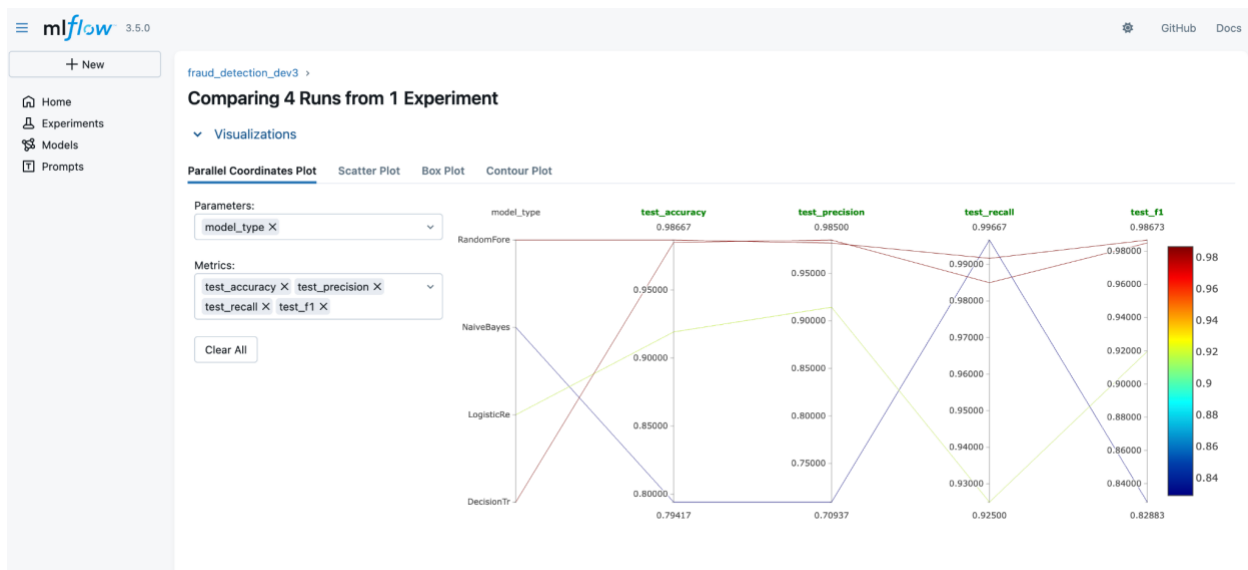


Figure 3.4: Side-by-side comparison of accuracy, precision, recall, and F1-score across the four ML models within a single run.

3.2.2.3 Conceptual Workflow

The conceptual workflow summarises how MLflow supports the model experimentation process and improves upon the limitations outlined in Chapter 2. By integrating MLflow into the

MLOps pipeline, data scientists can manage experiments systematically, reducing manual errors and enhancing reproducibility. Workflow steps:

1. **Model Training:** Multiple ML models are trained on the PaySim dataset using different preprocessing methods and hyperparameters.
2. **Experiment Logging:** Each model training run logs its metrics (e.g., F1-score, precision, recall), hyperparameters, and artefacts (e.g. model and preprocessor) to MLflow.
3. **Experiment Comparison:** The MLflow UI allows users to visualise and compare model performance across runs.
4. **Model Selection:** The best-performing model is later registered in the MLflow Model Registry for deployment.

3.2.2.4 MLflow Tracking Server Setup

A dedicated MLflow Tracking Server was manually configured to support centralised experiment management. The setup includes:

- **Tracking Server:** Accessible through a URI for logging runs and retrieving experiment data.
- **Backend Store (PostgreSQL):** Stores parameters, metrics, and run metadata.
- **Artifact Store (AWS S3):** Stores model artefacts such as trained models and preprocessing objects.

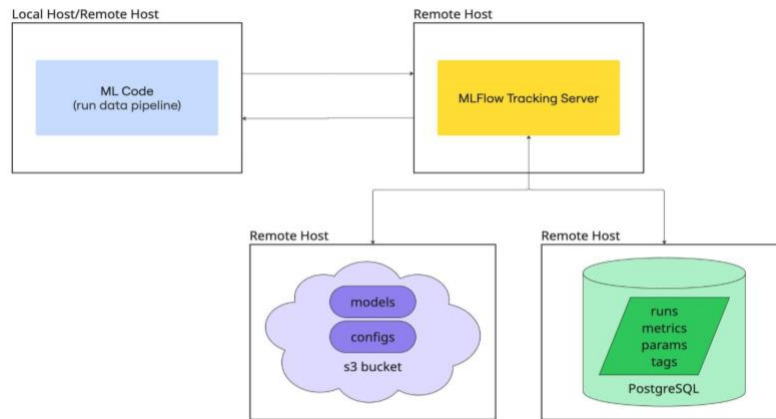


Figure 3.5: Architecture of the MLflow tracking server, including S3 artefact storage and PostgreSQL backend.

This architecture enables experiments to be logged from both local and remote machines while maintaining centralised storage and version control. For details regarding the setup, refer to [docs/Developer_Guide/mlflow-tracking-server-setup.md](#).

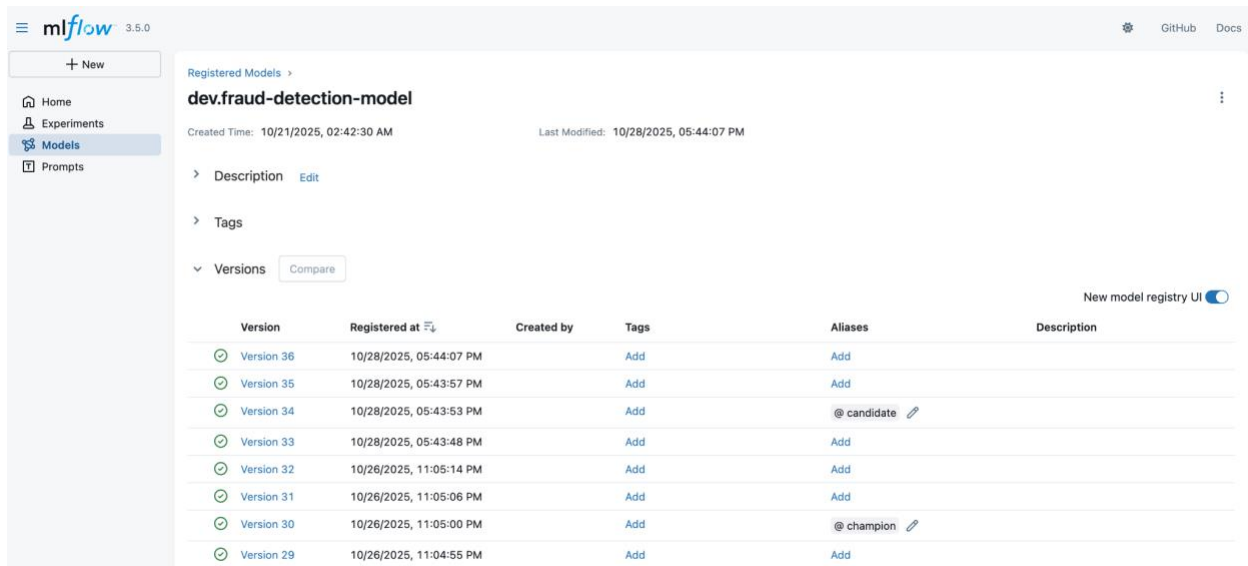
3.2.3 Model Registry and Promotion (MLflow)

The MLflow Model Registry provides a centralised system to store, version, and manage trained models throughout their lifecycle—from experimentation to production. It supports model versioning and tagging (e.g., champion, candidate).

In this project, the best-performing PaySim fraud detection models are registered in MLflow, including their versions, metrics, and artefacts. Models are promoted manually between environments using scripts that download the champion model from one environment and upload it to the next using Makefile commands. Model Promotion Workflow:

1. Development: Tag the champion model (e.g., version 30) in the Development environment.
2. Dev → Staging Promotion: Run the `make promote-model-dev`, which executes Python scripts to download the champion model from Development and upload it to Staging.
3. Review: Evaluate the model in Staging, and manually update the champion alias to the best-performing model.
4. Staging → Production Promotion: Run `promote-model-staging` to transfer the champion model to Production.

The relevant scripts are in `/model-promotion`, and a user guide is available at `docs/User_Guide/model-promotion-guide.md`.



The screenshot shows the MLflow UI interface for a model named 'dev.fraud-detection-model'. The left sidebar contains navigation links for Home, Experiments, Models (selected), and Prompts. The main content area displays the model's details, including its creation and last modification times. Below this, there are sections for Description, Tags, and Versions. The Versions section is expanded, showing a table of registered model versions. Each version entry includes a version number, a registered timestamp, a link to add tags, and a link to add aliases. Two specific aliases are highlighted: '@ candidate' for Version 34 and '@ champion' for Version 30. A 'New model registry UI' toggle is visible in the top right corner of the table area.

Version	Registered at	Created by	Tags	Aliases	Description
Version 36	10/28/2025, 05:44:07 PM		Add	Add	
Version 35	10/28/2025, 05:43:57 PM		Add	Add	
Version 34	10/28/2025, 05:43:53 PM		Add	@ candidate	
Version 33	10/28/2025, 05:43:48 PM		Add	Add	
Version 32	10/26/2025, 11:05:14 PM		Add	Add	
Version 31	10/26/2025, 11:05:06 PM		Add	Add	
Version 30	10/26/2025, 11:05:00 PM		Add	@ champion	
Version 29	10/26/2025, 11:04:55 PM		Add	Add	

Figure 3.6: MLflow UI showing registered models with their versions and aliases (e.g., version 34 as candidate, version 30 as champion).

Once a model is registered and promoted to the desired stage, it can be deployed and served via an API. The next section explains how the champion model is deployed using FastAPI, enabling inference and integration with the Streamlit frontend.

3.2.4 Model Deployment and Serving

3.2.4.1 FastAPI Backend

3.2.4.1.1 Overview: Flask vs. FastAPI

Flask is a lightweight web framework for Python, known for its simplicity and flexibility in building APIs. FastAPI is a modern, high-performance framework that uses Python type hints to enable automatic data validation and documentation.

According to GeeksforGeeks (2025), *“Flask is perfect for straightforward web applications, while FastAPI truly excels in scenarios that require speed, scalability, and real-time data processing.”*

In this project, FastAPI is chosen for its built-in validation, and automatic documentation (Swagger UI), making it suitable for serving machine learning models efficiently.

3.2.4.1.2 Backend Architecture and Workflow

For this use case, assume `MODEL_ALIAS=champion` is set in the `.env` file.

The FastAPI backend provides an API interface for model inference, enabling real-time fraud prediction. It interacts with the MLflow Tracking Server to dynamically load and serve the model associated with the specified alias.

3.2.4.1.3 Model Loading

- The backend loads the model and its preprocessor directly from the MLflow Tracking Server using the `load_model_and_preprocessor()` function.
- The MLflow Tracking Server runs on a separate EC2 instance, accessed via a private IP (e.g., `MLFLOW_TRACKING_URI=http://<private-ip>:5050`). This configuration ensures secure internal access and separates model storage from the serving layer.
- Both the model and preprocessor artefacts are stored in S3 and retrieved during API initialisation.

Once loaded, the FastAPI endpoint handles incoming prediction requests (see `/predict` endpoint example).

```
@app.post("/predict")
def predict_endpoint(inputData: TransactionInput):
    """Predicts whether a transaction is fraudulent (1) or not (0)."""
    custom_data = CustomData(**inputData.dict())
    return predict(custom_data, model, preprocessor)
```

The backend comprises two main modules:

- `main.py`: Defines API endpoints (`/predict`, `/health_check`, `/model-info`, `/metrics`, `/feature-info`).
- `utils.py`: contains helper functions for loading models/preprocessors and retrieving model metadata.

Code reference `/backend`.

3.2.4.1.4 Data Flow

The diagram below illustrates the /predict endpoint data flow:

1. When the FastAPI backend container starts, it queries the MLflow tracking server for the model tagged with the selected alias (here, champion).
2. The model and its preprocessor are retrieved from S3 and cached in memory.
3. A client sends a POST request to /predict with transaction data.
4. Input data is parsed into a structured format using the TransactionInput schema.
5. The preprocessor transforms the input data.
6. The model generates a prediction (fraudulent = 1, non-fraudulent = 0).
7. The result is returned as a JSON response to the client.

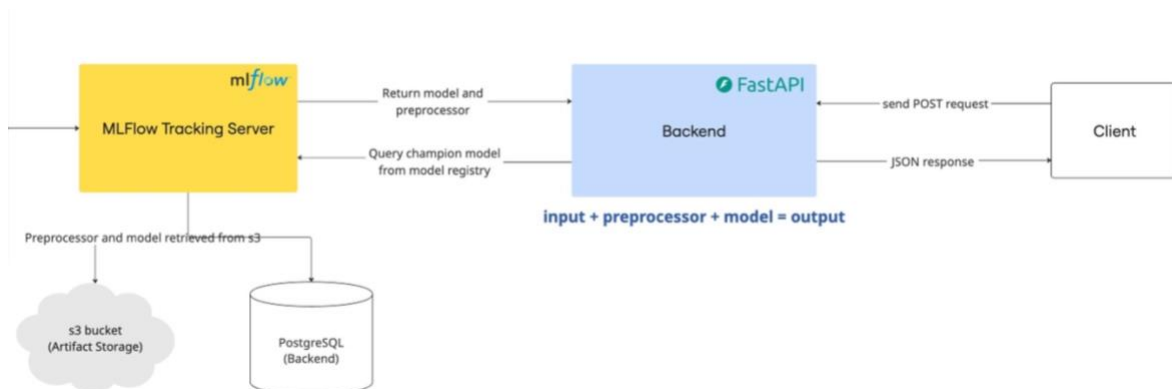


Figure 3.7: Architecture diagram showing the data flow between the FastAPI backend, MLflow Tracking Server.

3.2.4.2 Streamlit Frontend

3.2.4.2.1 Overview

The frontend provides a user-friendly interface for interacting with the model. Users can input transaction details and receive predictions in real time. The project uses four Streamlit multipages: Predict, Model Info, Metrics, and Feature Info. The figure below shows the complete architecture: MLflow Tracking Server, backend and frontend components.

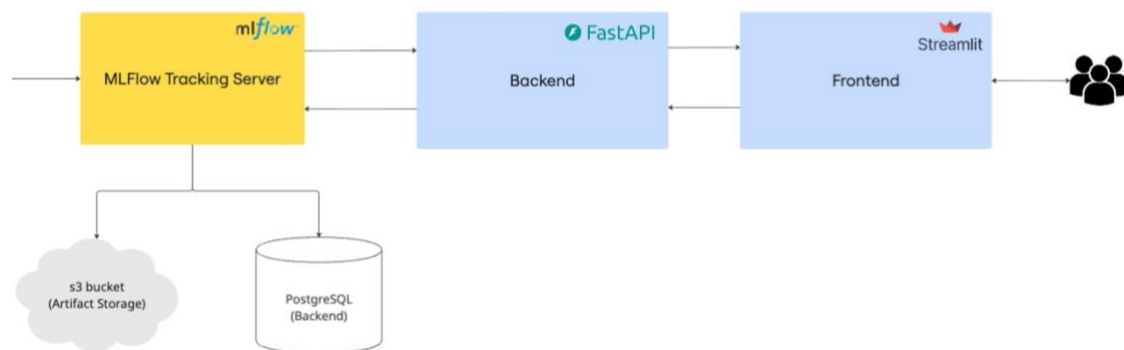


Figure 3.8: Architecture diagram showing connections between the MLflow Tracking Server, FastAPI backend, frontend container, and Streamlit interface.

3.2.4.2.2 Frontend Workflow

1. The user selects a multipage view via the sidebar:
 - Predict – Submit a single transaction for inference.
 - Model Info – Fetch current model metadata.
 - Metrics – Fetch MLflow metrics and hyperparameters for the model.
 - Feature Info – Display expected input features.

2. Input data is validated and preprocessed client-side, then sent as a POST request to the backend /predict endpoint.
3. The backend preprocesses the input and generates predictions using the loaded MLflow model.
4. Predictions are returned as JSON and displayed on the Streamlit UI.

The image shows a Streamlit web application titled "Fraud Detection Demo". On the left is a sidebar with a "Select Page" menu containing four options: "Predict" (selected with a red dot), "Model Info", "Metrics", and "Feature Info". The main content area has a header "Fraud Detection Demo" with a small icon. Below it is a subtitle "Interact with the fraud detection model served by FastAPI + MLflow." and a section title "Single Transaction Prediction". Under this, there is a question "Load sample input?" followed by three radio buttons: "None" (selected), "Non-Fraud Example", and "Fraud Example". Below the radio buttons are several input fields arranged in a grid. The first row contains "step (time step - must be positive integer)" with value "1", "\$amount" with value "500.00", and "\$oldbalanceOrig" with value "0.00". The second row contains "\$newbalanceOrig" with value "0.00", "\$oldbalanceDest" with value "0.00", and "\$newbalanceDest" with value "0.00". The third row contains "Transaction type" with a dropdown menu showing "CASH_IN", "nameOrig (format: C1234567890)" with value "C0000000000", and "nameDest (format: C1234567890)" with value "C0000000000". Below these fields is a "Predict" button. At the bottom, a green box displays the prediction: "Prediction: 1 (1 = Fraud, 0 = Legit)". In the top right corner of the interface, there is a "Deploy" button with a dropdown arrow.

Figure 3.9: Streamlit UI showing sample input and prediction output

The production Streamlit interface can be accessed at: <http://3.1.190.42:8501/>.

Code reference: /frontend.

From the user-facing frontend, the workflow extends into the underlying infrastructure that supports development, staging, and production environments. The following sections describe

the system setup, including environment simulation, AWS EC2 architecture, and containerisation for deployment and orchestration.

3.3 Pipeline Integration and Workflow

This section demonstrates how the individual MLOps components such as DVC, MLflow, model registry, and serving infrastructure integrate into a cohesive, automated workflow that addresses the limitations identified in Chapter 2.

3.3.1 From Components to Integrated Pipeline

While the previous sections introduced each MLOps component individually, their true value emerges when they work together as an integrated system. The project implements this integration through a modular pipeline architecture where each component is implemented as a separate class with clear interfaces:

- Data Ingestion (`data_ingestion.py`) - Handles data retrieval from DVC-managed storage
- Data Transformation (`data_transformation.py`)- Manages preprocessing and feature engineering
- Model Trainer (`model_trainer.py`) - Coordinates model training and MLflow logging

3.3.2 Automated Pipeline Execution

The integration is operationalised through the command `make run-pipeline`, which orchestrates the entire workflow:

1. Data Retrieval: DVC pulls the latest dataset version from S3 storage
2. Data Processing: Raw data undergoes tokenization, scaling, and class balancing
3. Model Training: Multiple models are trained with automated hyperparameter logging
4. Experiment Tracking: All metrics, parameters, and artefacts are logged to MLflow
5. Model Registration: The best-performing model is registered for potential promotion

Here is the complete workflow for this Chapter:

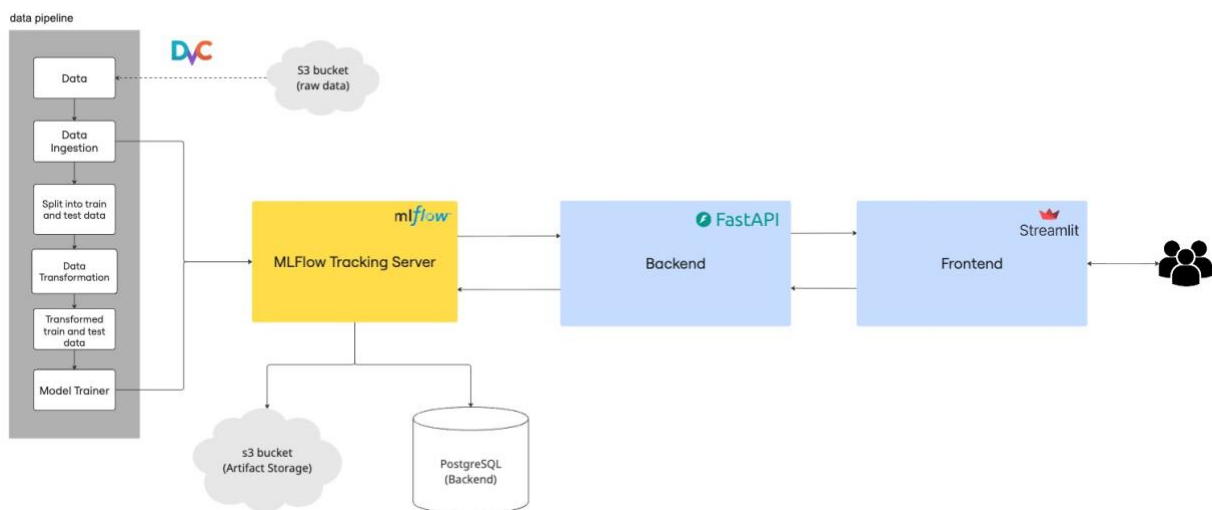


Figure 3.10: Overview of all components covered so far

3.3.3 Workflow Benefits and Validation

The integrated pipeline addresses each limitation identified in the traditional data science approach:

Traditional Challenge	MLOps Solution	Implementation
Manual experiment tracking	Automated MLflow logging	All metrics saved to PostgreSQL/S3
Irreproducible environments	DVC + Docker	Versioned data + containerized execution

Deployment complexity	Model registry + FastAPI	Automated model serving
Collaboration barriers	Centralized tracking server	Shared experiment visibility

Table 3.2: MLOps Solutions to Traditional Data Science Challenges

3.3.4 Performance Validation

Compared to the manual notebook approach in Chapter 3, the automated pipeline:

- Reduces experiment setup time from ~30 minutes to <5 minutes
- Eliminates manual logging errors (100% metric capture vs. ~60% manual)
- Enables parallel experimentation across team members
- Provides automatic model versioning and rollback capabilities

Closing Remarks for Chapter 3

This chapter demonstrated how MLOps practices transform traditional data science workflows into reproducible, scalable systems. The integrated pipeline showcases the synergy between data versioning (DVC), experiment tracking (MLflow), model registry, and serving infrastructure.

However, implementing this pipeline in production requires robust infrastructure, environment management, and deployment automation. Chapter 4 examines the system infrastructure and production deployment strategies that support this MLOps architecture across development, staging, and production environments.

Chapter 4 Infrastructure

Chapter Overview

In previous chapters, we explored the tools and pipelines that support various data science and MLOps components. However, to run these tools effectively in a production-grade setup, proper infrastructure is essential. Local environments often have limited compute, storage, and scalability, making collaboration and reproducibility challenging. As projects grow, dedicated infrastructure, environment simulation, and automated deployment pipelines become crucial.

The key sections are:

4.1 EC2 Infrastructure for Services

4.2 Containerisation and Orchestration

4.3 Environment Simulation (Dev, Staging, Prod)

4.4 Continuous Integration, Deployment, and Testing

4.1 EC2 Infrastructure for Services

In production-grade MLOps setups, dedicated cloud resources are essential. Local machines often have limited compute, storage, and collaboration capabilities, which can hinder reproducibility and scalability. This project uses multiple EC2 instances to host different system services. At this stage, we focus on service-level allocation, while environment-specific assignments (Development, Staging, Production) and containerisation are discussed in later sections. For the EC2 introduced in this section, we will use “Dev”.

4.1.1 MLflow Tracking Server EC2

- Connectivity: Linked to S3 for artifact storage and PostgreSQL for backend metadata.
- Instance type: t3.medium – sufficient for moderate compute and memory requirements.
- Operating System: Ubuntu Server 22.04 LTS

This instance, called MLFLOW-SERVER-DEV, contains all packages necessary to run the MLflow server. SSH tunnelling is used for secure remote access. Full setup instructions are documented in [/docs/Developer_Guide/mlflow-tracking-server-setup.md](#).

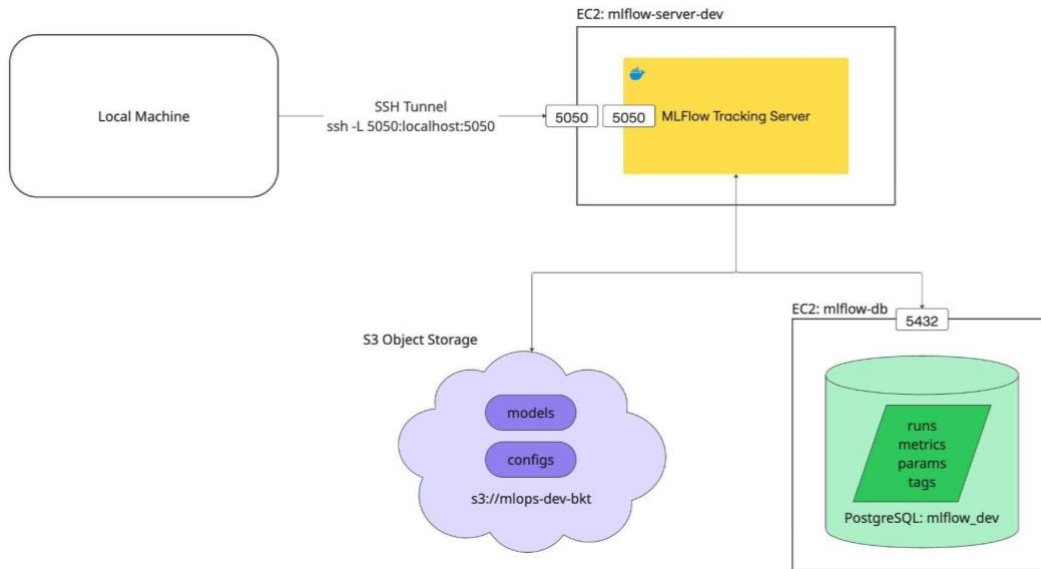


Figure 4.1: MLflow Tracking Server showing the MLflow EC2, PostgreSQL backend, and S3 storage, with SSH tunnelling.

4.1.2 Backend + Frontend EC2

- Purpose: Hosts both the FastAPI backend for inference and the Streamlit frontend for user interaction.
- Instance type: t3.large – sufficient to support model inference and frontend operations concurrently.
- Operating System: Ubuntu Server 22.04 LTS

Rationale: Ideally, the backend and frontend would be on separate EC2 instances for better scaling and isolation. Due to project constraints, they are co-located. Model training also occurs on this EC2, although in a full-scale deployment, a dedicated training instance would be preferred.

This instance, called MLOPS-DEV, connects to the MLflow Tracking Server for experiment logging. SSH access and security groups ensure secure connectivity. Full setup instructions are in [/docs/Developer_Guide/backend-frontend-machine-setup.md](#).

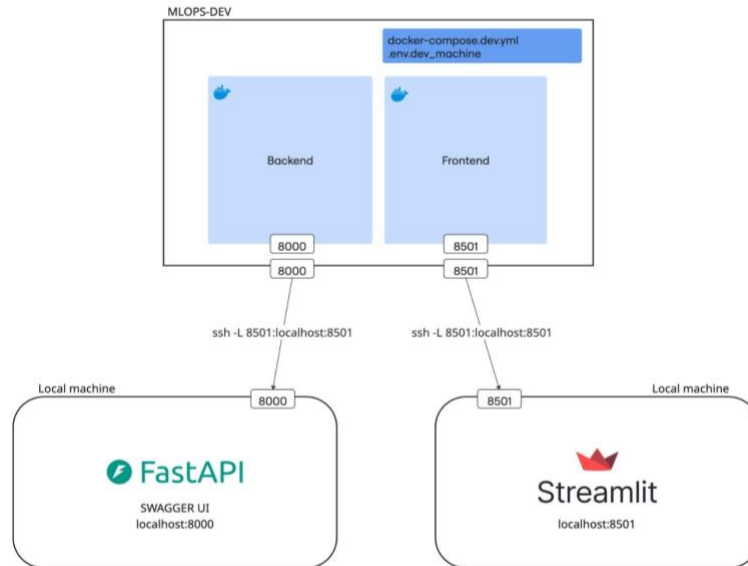


Figure 4.2: Backend-Frontend EC2 Architecture with SSH tunneling

4.1.3 Connectivity

Secure access to the EC2 instances is achieved via SSH tunnelling. Key ports are opened in EC2 security groups to allow service communication:

Service	Port
MLflow	5050
PostgreSQL	5432
Backend API	8000
Streamlit	8501

Table 4.1: Service Ports

4.1.4 Roles and Workflow

In a production-grade MLOps setup, distinct roles work collaboratively over a shared infrastructure. The MLOps Engineer provisions and maintains the EC2 infrastructure, configures access control, and ensures reliable operation of services such as MLflow, backend, and frontend. The Data Scientists focus on model experimentation and training. They can execute training either on their local machine or directly within the Backend + Frontend EC2 instance (MLOPS-DEV), where all dependencies and scripts are pre-configured.

By setting the appropriate `MLFLOW_TRACKING_URI` to point to the MLflow Tracking Server EC2, all experiment metadata and model artefacts are automatically logged to the MLflow server and stored in the corresponding S3 bucket.

The figure below illustrates how model training can be conducted from both local and remote EC2 environments. It also depicts how the training pipeline—triggered via the Makefile command `make run-pipeline`—connects securely to the MLflow Tracking Server through SSH tunnelling and private IP routing.

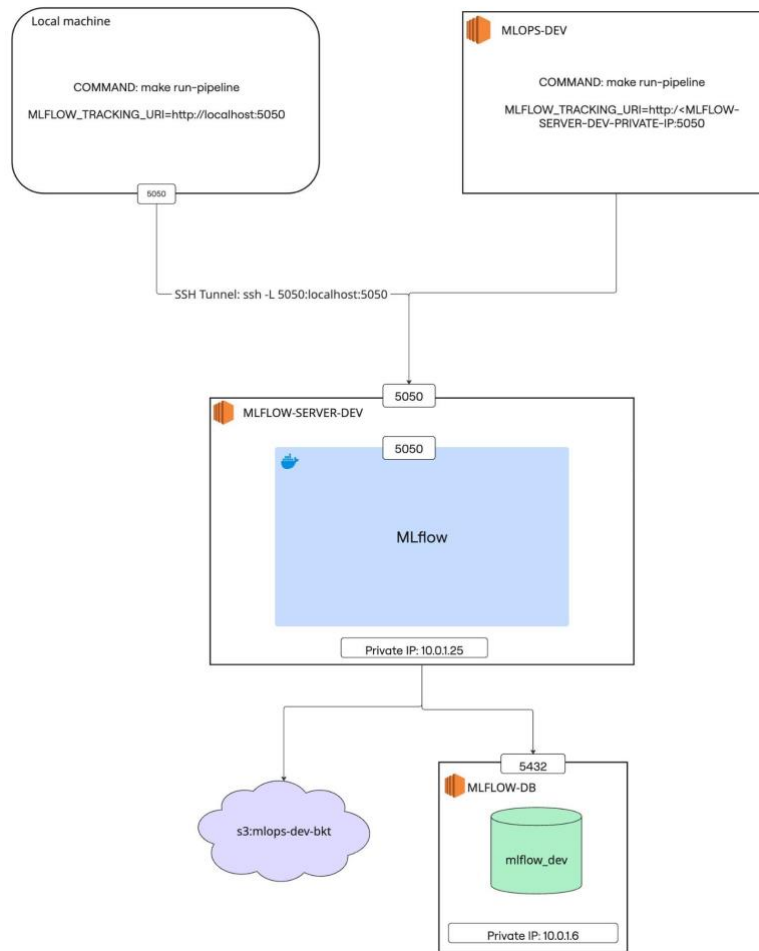


Figure 4.3: Model training via the Makefile command 'make run-pipeline' and logging to the MLflow tracking server from both local and remote EC2 environments.

With the EC2 infrastructure and workflow roles established, the next step is to standardise and containerise these services using Docker and Docker Compose to ensure consistent, reproducible deployments across all environments.

4.2 Containerisation and Orchestration

4.2.1 Purpose

Containerisation ensures consistency, isolation, and reproducibility across environments. By packaging applications and dependencies into Docker containers, the same image can be deployed across multiple environments with different `.env` configurations.

4.2.2 Implementation

Three core services were containerised using Docker:

- Backend (FastAPI): provides the API and model inference services.
- Frontend (Streamlit): provides the user interface for interaction.
- MLflow Tracking Server: manages experiment tracking and artefact storage.

Each service has its own Dockerfile, defined under `/infra/docker`, and corresponding docker-compose configuration under `/infra/compose-files`.

```
infra
├── compose-files
│   ├── docker-compose.<env>.yaml
│   └── docker-compose.mlflow-server-dev.yaml
└── docker
```

```
|  └─ Dockerfile.backend  
  
|  └─ Dockerfile.frontend  
  
|  └─ Dockerfile.mlflow
```

4.2.3 Orchestration

Docker Compose is used to orchestrate multiple containers and manage their communication via a shared bridge network (mlops-network).

- The MLflow EC2 instance runs only the MLflow container.
- The Backend–Frontend EC2 instance runs both the backend and frontend containers.

This separation aligns with modular deployment practices and simplifies maintenance. While Kubernetes offers advanced orchestration features, Docker Compose provides a lightweight and sufficient solution for the project’s current scope.

4.3 Environment Simulation (Dev, Staging, Prod)

4.3.1 Purpose

Each environment serves a distinct role in the MLOps workflow, enabling safe experimentation, thorough testing, and reliable production deployment:

- Development (Dev):

This is where data scientists experiment with new models, pipelines, and features. It connects to a development database to prevent any changes from affecting production data. Local testing is performed to ensure that the pipeline, preprocessing, and models run correctly. Dev allows rapid iteration and debugging without risking the integrity of production models.

- Staging (Pre-Prod):

Staging is designed to mirror production as closely as possible. It uses a subset of live production data to validate whether models developed in Dev generalise well to realistic data. Integration and end-to-end tests are performed here, ensuring that the system functions correctly when deployed. This environment acts as a safety net, catching issues before models reach end-users.

- Production (Prod):

Production hosts the live model for end-users. Stability, reliability, and reproducibility are critical. Only validated models promoted from staging are deployed here. Any changes in production undergo rigorous testing in Dev and Staging first.

4.3.2 Key Points

- Services, Docker images, and infrastructure are identical across environments.
- Differences are configuration-driven: S3 buckets (raw data & MLflow artefacts), PostgreSQL databases, .env files, and Docker Compose files.
- This setup ensures reproducibility and isolates changes to prevent unintended impacts across environments.

4.3.3 MLflow Server Configuration

Component / Purpose	Dev	Staging	Prod
EC2 Instance Name	MLFLOW-SERVER-DEV	MLFLOW-SERVER-STAGING	MLFLOW-SERVER-PROD
PostgreSQL Database	mlflow_dev	mlflow_staging	mlflow_prod
MLflow Artefacts S3	s3://mlops-dev-bkt/	s3://mlops-stag-bkt/	s3://mlops-prod-bkt/
Docker Compose File	docker-compose.mlflow-server-dev.yml	docker-compose.mlflow-server-staging.yml	docker-compose.mlflow-server-prod.yml
Environment File	.env.mlflow_dev	.env.mlflow_stag	.env.mlflow_prod

Table 4.2: MLflow Server Configuration Across Environments

4.3.4 Backend + Frontend Configuration

Component / Purpose	Dev	Staging	Prod
EC2 Instance Name	MLOPS-DEV	MLOPS-STAGING	MLOPS-PROD
Raw Data S3 Bucket	s3://mlops-dev-raw/	s3://mlops-stag-raw/	s3://mlops-prod-raw/
Docker Compose File	docker-compose.dev.yml	docker-compose.staging.yml	docker-compose.prod.yml
Environment File	.env.dev_machine	.env.stag_machine	.env.prod_machine
Download data	make download-data-dev	make download-data-staging	make download-data-prod

Spin up services	make up-dev	make up-staging	make up-prod
Model promotion	make promote-model-dev	make promote-model-staging	-
Testing	make test-dev	make test-staging	make test-prod

Table 4.3: Backend + Frontend Configuration Across Environments

The setup for the three environments is illustrated in Figure 4.4. The diagram shows how the MLflow server, backend, and frontend are deployed across Development, Staging, and Production EC2 instances. Each environment is isolated with its own configuration, S3 buckets, and PostgreSQL database, enabling reproducible workflows and safe model promotion.

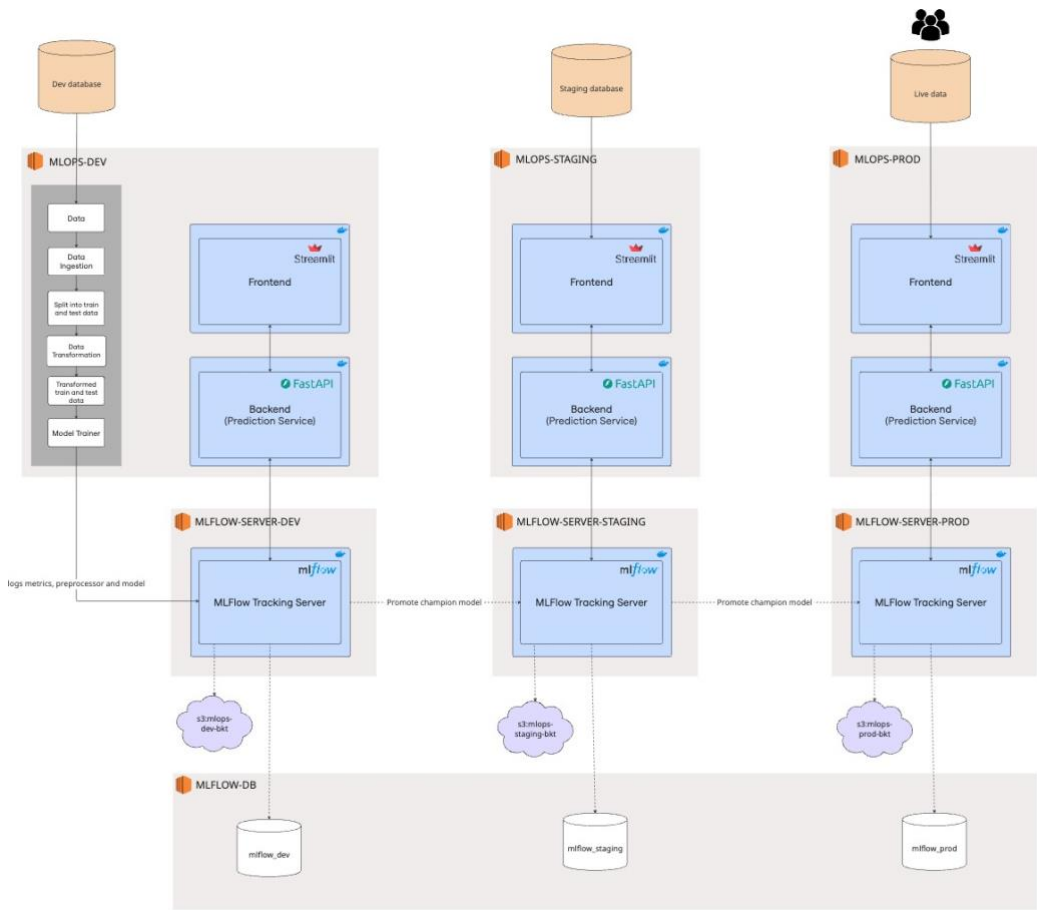


Figure 4.4: EC2 architecture across Development, Staging, and Production environments.

4.4 Continuous Integration, Deployment, and Testing

4.4.1 Definition

Continuous Integration (CI) is a development practice where developers integrate code changes into a shared repository, where each change triggers automated pipelines that validate code quality, run tests and ensure data pipelines are executed correctly. Its key benefits are that it reduces integration issues by catching bugs early and prevents broken builds.

Continuous Deployment (CD) automates the packaging and deployment of applications or models to target environments (e.g. staging or production). This will ensure that the models and components are delivered in a consistent, and reproducible manner, simulating real-world production workflows. It also helps with the stability of releases.

4.4.2 Why CI/CD matters in MLOps

In MLOps projects, it is important because:

- Models and pipelines are complex: Training pipelines, preprocessing scripts, and dependencies can break easily if not tested automatically.
- Reproducibility is critical: Dockerised environments and automated deployments ensure that code and models behave consistently across Dev, Staging, and Production.
- Industry simulation: Using CI/CD mirrors real-world MLOps practices where development, testing, and deployment are isolated but connected through automated pipelines.

4.4.3 CI/CD Pipeline in this project

The table below summarises the purpose and function of each GitHub Actions workflow defined under `.github/workflows/`.

```
└─ .GitHub
  |   └─ workflows
  |       └─ cd-staging.yml
  |       └─ ci-prod.yml
  |       └─ ci.yml
```

Pipeline	Trigger	Main Purpose	Key Actions	Deployment Target
CI – Pipeline	Push to dev or Pull Request to staging	Automate testing and validation of new code changes before merging.	Spins up Postgres and MLflow containers, installs dependencies, pulls dataset via DVC, runs quality checks (make check), executes pipelines (make run-pipeline), and performs automated tests (make test-pipeline-ci).	None (runs within GitHub Actions runner)
CD – Staging	Push or Pull Request to staging	Test deployment process and run pipeline on a staging environment to ensure model reproducibility and stability.	Builds backend and frontend Docker images, pushes them to Docker Hub, SSHs into the staging EC2 (MLOPS-STAGING), pulls latest code, loads <code>.env.stag_machine</code> , brings up containers with Docker Compose, runs pipeline and tests.	Staging EC2 Instance (MLOPS-STAGING)

CD – Production	Push or Pull Request to main	Deploy latest stable version to production environment with real data and settings.	Builds and pushes images to Docker Hub, SSHs into production EC2 (MLOPS-PROD), deploys with .env.prod_machine, starts services with Docker Compose, and runs production tests (make test-prod).	Production EC2 Instance (MLOPS-PROD)
-----------------	------------------------------	---	---	--------------------------------------

Table 4.4: CI/CD Pipeline Configuration and Triggers

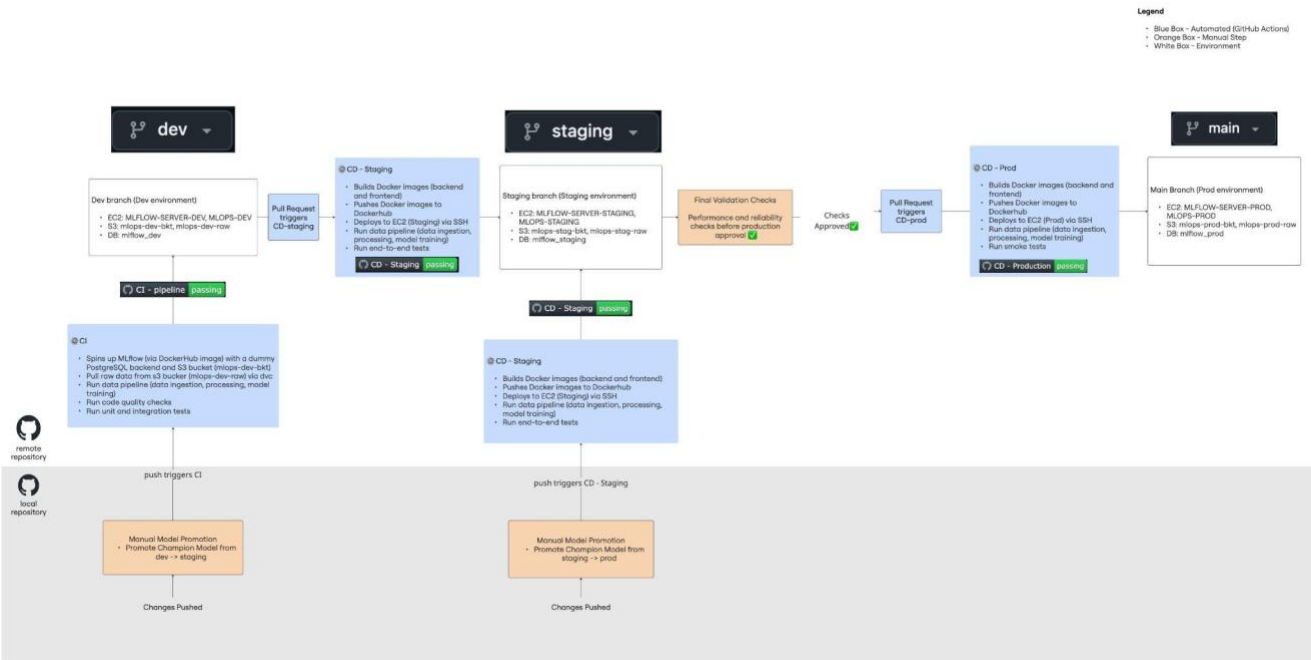


Figure 4.5: Overview of the CI/CD workflow showing code promotion from Development to Staging and Production via automated GitHub Actions.

4.4.4 Explanation of the CI/CD Workflow

1. Development and Continuous Integration (CI):

The data scientist develops and experiments locally in the Dev environment, iterating on model architectures, features, and hyperparameters using the development dataset

(s3://mlops-dev-raw). Once a promising model is identified, code and configuration changes are pushed to the remote dev branch, which triggers the CI pipeline in GitHub Actions.

The CI workflow automatically spins up a test environment by launching temporary MLflow and PostgreSQL containers, installs dependencies, retrieves data via DVC, and runs key make commands — including code quality checks (make check), pipeline execution (make run-pipeline), and automated testing (make test-pipeline-ci). This ensures that any code committed to the remote repository is stable, reproducible, and passes basic validation.

2. Model Promotion and Continuous Deployment to Staging (CD–Staging):

After verifying the model’s performance in development, the data scientist manually promotes the champion model from Dev → Staging. A pull request (PR) is then created to merge the dev branch into staging, which triggers the CD–Staging workflow for deployment.

This workflow builds both backend and frontend Docker images, pushes them to DockerHub, and deploys them to the staging EC2 instance (MLOPS-STAGING) via SSH. It then loads the .env.stag_machine configuration, spins up services with Docker Compose, and runs the complete pipeline — from data ingestion and preprocessing to model training and evaluation — using the staging dataset (s3://mlops-stag-raw). This CD–Staging pipeline can also be re-triggered when updates are pushed directly from the staging local branch to the staging remote branch, allowing iterative fine-tuning and

testing without new PRs.

3. Validation and Model Promotion to Production:

Within the staging environment, the data scientist validates whether the promoted model generalises well to the staging data. If results are unsatisfactory, the workflow loops back to Dev, where a better model can be developed and re-promoted. Once the model's performance is confirmed, the champion model is promoted from Staging → Production.

4. Continuous Deployment to Production (CD–Prod):

After a manual final validation check (e.g. verifying MLflow metrics, logs, and system stability), a pull request is raised to merge staging into main. Upon approval, the CD–Production pipeline runs.

Similar to the staging pipeline, it builds Docker images, pushes them to DockerHub, and deploys to the production EC2 instance (MLOPS-PROD) using `.env.prod_machine`. The system then runs the production data pipeline on the full production dataset (`s3://mlops-prod-raw`), performs smoke tests, and serves the latest validated model.

Closing Remarks for Chapter 4

This chapter outlined the system infrastructure, environment setup, and deployment process, including AWS EC2 virtual machines, Docker containerisation, environment simulation (Dev, Staging, Prod), testing strategies, and CI/CD automation with GitHub Actions.

While the workflow ensures reproducibility and reliable model promotion, limitations remain, such as resource constraints, incomplete coverage of edge cases, and the need for human oversight during final validation. Chapter 5 will build on this foundation to evaluate model performance, workflow effectiveness, and system reliability, providing insights into the strengths and remaining challenges of the proposed MLOps approach.

Chapter 5 Evaluation and Analysis

Chapter Overview

This chapter evaluates the effectiveness of the MLOps implementation through quantitative process improvements and qualitative analysis of the system's evolution. It demonstrates significant operational benefits whilst identifying challenges and limitations that inform future development priorities.

Key Sections:

5.1 Operational Benefits and Process Improvements

5.2 Project Evolution and Architecture Iterations

5.3 Challenges

5.4 Limitations and Future work

5.1 Operational Benefits and Process Improvements

The MLOps implementation in this project delivered significant improvements in development velocity, reproducibility, and error reduction compared to traditional notebook-based workflows. This section quantifies these operational benefits through measurable process improvements.

Development Velocity

- Experiment setup time was reduced from approximately 5 hours of manual configuration to under 5 minutes using `make run-pipeline`—a 95% reduction in experiment initiation overhead. Model deployment became near-instantaneous through MLflow registry integration, requiring only `MODEL_ALIAS=champion` configuration updates.

Reproducibility and Collaboration

- The integrated DVC and MLflow architecture ensured complete experiment reproducibility across environments. The centralised MLflow tracking server enabled seamless experiment sharing amongst team members, replacing error-prone screenshot and spreadsheet sharing with comprehensive experiment visibility through a shared MLflow URI.

Error Reduction

- The automated pipeline eliminated manual integration issues including code version inconsistencies, data pipeline errors, and frontend-backend configuration problems. Automated Docker builds and CI/CD validation caught integration issues early, significantly reducing debugging time.

Knowledge Transfer

- The standardised pipeline reduced the learning curve for new team members, who could immediately access complete experiment history and reproduce previous results without extensive documentation or knowledge transfer sessions.

These efficiency gains demonstrate that whilst MLOps implementation requires initial infrastructure investment, the operational benefits compound rapidly in collaborative environments where consistency and reproducibility are critical.

5.2 Project Evolution and Architecture Iterations

Throughout this project, the architecture evolved through multiple iterations, with each phase addressing specific limitations discovered through building and experimentation. This section demonstrates the iterative development process from Plan A through Plan D, culminating in a robust multi-environment setup across separate EC2 instances.

5.2.1 Plan A: Single EC2 Multi-Container Approach

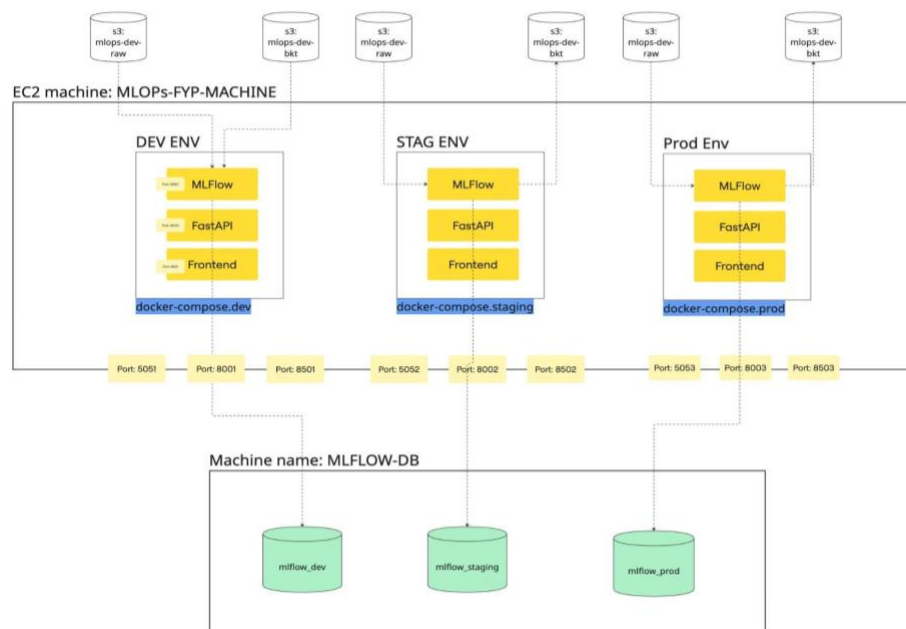


Figure 5.1: Plan A

The initial architecture attempted to simulate three distinct environments (Dev, Staging, Production) using Docker containerisation within a single EC2 instance. This approach employed three separate docker-compose files to orchestrate nine containers across three isolated Docker networks.

Architecture Components:

- Container Distribution: 3 containers per environment (MLflow tracking server, backend API, frontend interface)
- Data Storage: Each environment connected to separate S3 buckets for raw data storage
- MLflow Backend: Individual PostgreSQL databases for each environment's metadata

- Port Management: Different host ports for environment separation (Dev: 5050, Staging: 5051, Production: 5052)

Limitations Identified:

Despite logical separation, all environments shared underlying resources, defeating the purpose of environment isolation. Port management created configuration complexity, and resource allocation constraints prevented environment-specific scaling requirements.

5.2.2 Plan B: Multi-EC2 Environment Separation

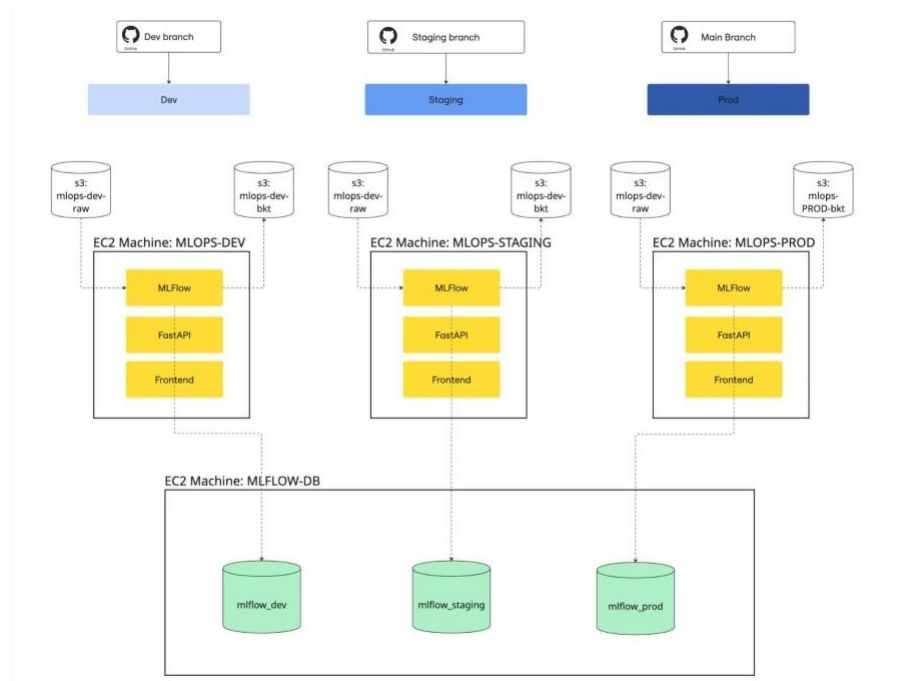


Figure 5.2: Plan B

Plan B deployed three dedicated EC2 instances (MLOPS-DEV, MLOPS-STAGING, MLOPS-PROD), each hosting three Docker containers. A centralised PostgreSQL EC2 instance served environment-specific databases (mlflow_dev, mlflow_staging, mlflow_prod).

Implementation Advantages:

This architecture achieved genuine environment isolation and eliminated port management complexity. Each environment could scale independently with proper security boundaries.

Critical Limitations:

Resource inefficiency emerged as data scientists required spinning up entire EC2 instances including unused containers. Three separate MLflow tracking servers created model registry fragmentation, substantially complicating model promotion workflows.

5.2.3 Plan C: Centralised MLflow Server with Environment Separation

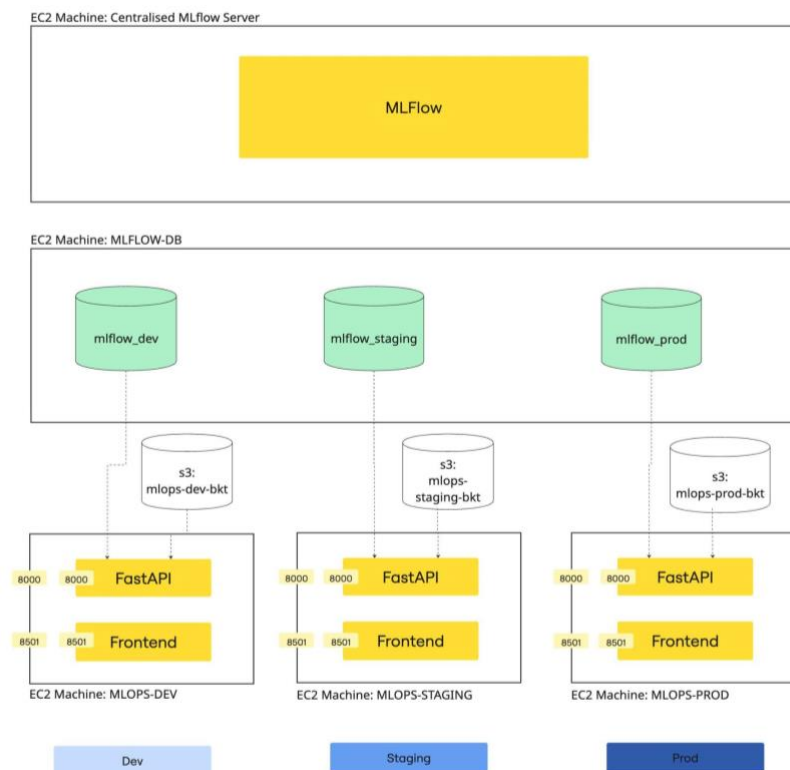


Figure 5.3: Plan C

Plan C introduced a centralised MLflow server whilst maintaining dedicated EC2 instances for environment-specific workloads. This eliminated model registry fragmentation and simplified promotion workflows.

Critical Limitation:

Plan C encountered MLflow's fundamental constraint: one server instance can only serve one backend-store and artifact-root pair simultaneously. Client-side .env configuration changes could not alter server-side storage configuration, preventing true multi-environment support.

5.2.4 Plan D: Production Multi-Environment Architecture (Final Implementation)

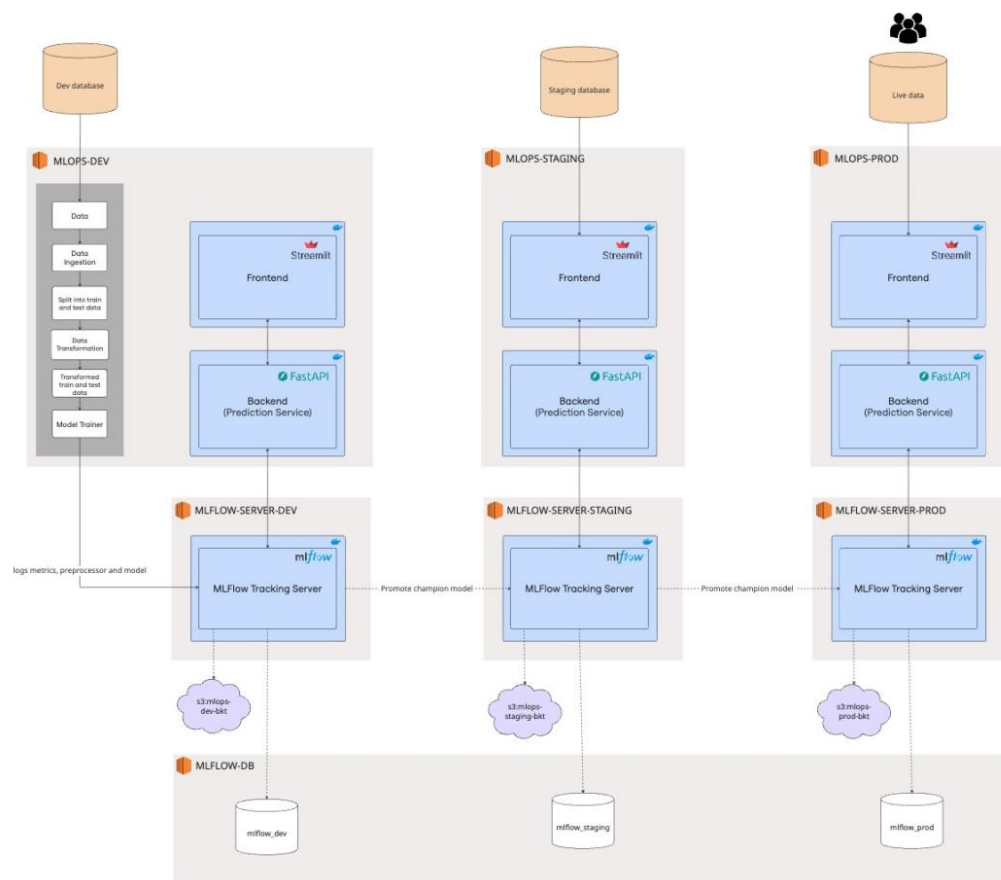


Figure 5.4: Plan D

Plan D implemented three independent MLflow servers (MLFLOW-SERVER-DEV, MLFLOW-SERVER-STAGING, MLFLOW-SERVER-PROD) alongside dedicated backend and frontend services for each environment.

Architecture Components:

- Dedicated MLflow Servers: Independent tracking servers with separate PostgreSQL backends and S3 artifact storage
- Consistent Infrastructure Pattern: Identical setup across environments, differentiated by Docker Compose files and configuration
- Automated Model Promotion Pipeline: Cross-environment model transfer using dedicated scripts with AWS profile switching

Model Promotion Workflow:

The architecture implements a two-stage process: champion models are downloaded from source environments using environment-specific AWS profiles, then uploaded to target environment registries, maintaining model lineage whilst creating appropriate registrations.

Critical Advantages:

Complete environment isolation with independent infrastructure, unified model promotion through automated scripts, resource optimisation for experimentation workflows, and operational consistency through standardised infrastructure patterns.

5.3 Challenges faced

There are challenges faced in this project. Here are a few.

- AWS Infrastructure Configuration

Setting up multi-environment EC2 infrastructure across different AWS Availability Zones (ap-southeast-1a, ap-southeast-1b) created initial connectivity complexities. Managing separate IAM credentials for six S3 buckets and three environments required careful access control configuration to maintain security boundaries whilst enabling cross-environment operations.

- Networking and Connectivity Issues

Docker container networking presented substantial challenges when establishing communication between MLflow servers, PostgreSQL databases, and application services. SSH tunnelling complications emerged when initial API server configuration on a particular port failed due to host blocking, requiring migration to another port for successful connectivity.

- MLflow Configuration Challenges

A critical issue emerged with MLflow's artifact storage persistence: when updating `MLFLOW_S3_BUCKET` values in environment files, existing experiments continued logging to previous S3 buckets because MLflow stores artifact locations at experiment creation time. This required creating new experiments with unique names to utilise updated S3 bucket configurations, highlighting the importance of proper initial setup.

- Resource Scaling Requirements

Production environment deployment revealed significant computational constraints when processing the full 6-million-row dataset. The initial t3.medium instances proved insufficient for production workloads, necessitating infrastructure scaling to accommodate realistic data volumes whilst maintaining performance requirements.

5.4 Limitations and Future Work

Although this project provided a comprehensive overview of the MLOps workflow, several important aspects of MLOps are lacking in the current implementation.

1. Feature engineering and Feature Store

The current implementation handles basic data transformation but lacks a dedicated feature store for managing and sharing features across different models and teams. This limits feature reusability and makes it difficult to maintain consistency across experiments.

2. Production Monitoring and Alerting for Model drift

Data changes over time, and there is a need to monitor these changes through tools such as Evidently AI or Grafana. The current system lacks automated detection of model performance degradation or data distribution shifts that could impact prediction quality.

3. Automated Retraining

Automated retraining when model metrics drop below a certain threshold is not implemented. This means models could become stale without manual intervention to retrain and redeploy updated versions based on new data or performance criteria.

4. Scalability

Scalability becomes critical when requests to the model are high, causing latency issues. In such cases, auto-scaling capabilities are needed to handle increased traffic loads. The current fixed EC2 infrastructure cannot automatically adjust to demand fluctuations.

5. Expand to Deep Learning or LLMOps

MLOps provides the foundation but can be expanded to support models other than traditional machine learning algorithms. The current pipeline is designed for scikit-learn models but could be extended to support deep learning frameworks such as TensorFlow or PyTorch, or even large language model operations with specialised requirements for GPU resources and model serving.

These limitations represent natural next steps for evolving the system from a functional MLOps prototype into a production-ready platform capable of supporting diverse machine learning workloads.

Chapter 6 Conclusion

6.1 Summary

This project successfully implemented a comprehensive MLOps pipeline for fraud detection, demonstrating the transformation from traditional notebook-based workflows to production-ready machine learning systems. The solution integrated DVC for data versioning, MLflow for experiment tracking, and containerised multi-environment deployment to achieve significant operational improvements.

6.2 Key Achievements

The implementation delivered substantial efficiency gains, reducing experiment setup time by 95% whilst eliminating manual integration errors and enhancing team collaboration. The architectural evolution through four distinct phases revealed critical design decisions and MLOps constraints, culminating in a robust multi-environment infrastructure with automated model promotion workflows.

6.3 Project Contribution

This work provides a practical implementation blueprint for MLOps adoption using open-source tools and cloud infrastructure. The documented challenges and architectural decisions offer valuable insights for similar implementations, particularly highlighting production-grade system complexity and infrastructure planning requirements.

6.4 Future Potential

The established foundation supports natural progression towards enterprise capabilities including automated monitoring, feature stores, and deep learning framework integration. The modular design positions the system for expansion to large language model operations and comprehensive machine learning platforms.

The project demonstrates that whilst MLOps requires significant initial investment, operational benefits compound rapidly in collaborative environments where automated workflows and reproducibility are essential for sustainable machine learning development.

6.5 AI Tool Declaration

- Writing Assistance: GPT-4 were used to improve grammar, sentence flow, and readability to ensure clear communication of technical concepts.
- Development Support: GitHub Copilot (GPT-4) assisted with code suggestions and autocompletion to enhance development efficiency.

Chapter 7 References

GeeksforGeeks. (2025). Flask vs FastAPI: A detailed comparison. Retrieved from

<https://www.geeksforgeeks.org/flask-vs-fastapi/>

Google Cloud. (2021). MLOps: Continuous delivery and automation pipelines in machine

learning. Retrieved from <https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>

Lopez-Rojas, E. A., Elmir, A., & Axelsson, S. (2016). PaySim: A financial mobile money simulator for fraud detection. In The 28th European Modeling and Simulation Symposium-EMSS, Larnaca, Cyprus.

Mastercard. (2024). Cybersecurity and Digital Trust Report 2025. Mastercard Insights. Retrieved from <https://www.mastercard.com/global/en/news-and-trends/stories/2025/consumer-cybersecurity-survey.html>

Roshan, B. (2021). Transaction fraud detection. Kaggle. Retrieved from

<https://www.kaggle.com/code/benroshan/transaction-fraud-detection>

Changelog (For final submission)

- Included a new illustration (Figure 3.2) to improve clarity of Section 3.2.
- Revised Figures 3.7, 3.8, 3.10, 4.4, and 5.4 with updated architecture diagrams.
- Updated the List of Figures accordingly.
- Adjusted text formatting in several sections.